



Double

🕒 Created time	July 25, 2026 8:07 AM
☰ Category	CommonVariables
👤 Last edited by	 R.A R.A
🕒 Last updated time	July 25, 2026 9:03 AM

```
Console.WriteLine("Hello User! Please write 2 numbers.");
string? numberOneText = Console.ReadLine();
string? numberTwoText = Console.ReadLine();

bool isNumberOneValid = int.TryParse(numberOneText, out int
numberOne);
bool isNumberTwoValid = int.TryParse(numberTwoText, out int
numberTwo);

Console.WriteLine($"This is valid: {isNumberOneValid}, the
number was {numberOne} and this is valid: {isNumberTwoVali
d}, the number was {numberTwo}");

if (isNumberOneValid && isNumberTwoValid)
{
    int sum = numberOne + numberTwo;
    int difference = numberOne - numberTwo;
    double average = (double)numberOne / numberTwo;

    Console.WriteLine($"Sum: {sum}, Difference: {differenc
e}, Average: {average}");
}

//~~~~~
Doubles can have a Decimal point - 1.23, 43
```

```
double averageAge;

averageAge = (43.0 + 22 + 62) / 3;

averageAge = 2.3 / 3;

Console.WriteLine(averageAge);
```

ה-Double

זה קצת שונה מ-**Int**, כי ל-**Double** יכולה להיות נקודה עשרונית. כמו **1.23** ו-**43**, אנחנו שואלים למה **43**, לזה אין נקודה עשרונית. התשובה היא:

טכנית זה יכול להיות עם נקודה עשרונית - **43.0**.

אבל בכל מקרה זה בסדר, זה לא חייב להיות עם נקודה עשרונית, זה לא ננעל לנקודה עשרונית, אבל אנחנו יכולים להיות עם נקודה עשרונית ב-**Double** שלנו. התוצאה של הקוד הזה תהיה ב-**Int**:

```
averageAge = (43 + 21 + 62) / 3;
```

אבל אם נכתוב את אותו קוד ככה:

```
averageAge = (43.0 + 21 + 62) / 3;
```

זה עכשיו **Double**, התוצאה של הקוד הזה תהיה

```
Double = 42.333333333333336
```

ה-Int

מניחים שהם מספר שלם אם הם מספר שלם, אבל אם נוסף **0.0 (43.0)** שזה עדיין אותו מספר, הקומפיילר רואה את זה כמו **Double** בסוג, והקומפיילר יראה את שאר המספרים ואת התוצאה שלהם כ-**Double**, אז אנחנו צריכים לתת לקומפיילר רמז אם אנחנו רוצים **Double**.

שימו לב שמספר שלם מסוג **Integer** יכול להיכנס ל-**Double** בלי בעיה, כי זה הולך מפחות ספציפי ליותר ספציפי, אז **43** יכול להתאים ל-**Integer** עם מספר שלם, ו-**43** יכול להתאים ל-**Double** בלי לשנות שום דבר כי **43** זה **43**, אבל אם היה לנו הכיוון ההפוך שמנסה לשים

Double ב-**Integer**, כמו המספר הזה **1.23**, היינו צריכים לחתוך את ה-**23** ממנו וזה לא היה עובד כי היינו מאבדים נתונים, לכן אנחנו לא יכולים לשים **Double** בתוך **Integer** גם אם ה-**Double** הוא רק **43**, כי המערכת לא יודעת שזה תמיד יהיה המקרה. העיקר כאן הוא ש-**Double** יש להם נקודות עשרוניות בהם, או שיכולות להיות להם נקודות עשרוניות, ואם מוסיפים **3** מספרים כמו

ומחלקים ב-3 $43 + 21 + 62$

אז כיוון שכולם מונחים כ-**Integer** על ידי הקומפיילר, התוצאה תהיה בערך **Integer** שאז כן נכנס ל-**Double** אבל זה מאבד את נקודת העשרון שלנו בחלוקה הזו בגלל שימוש ב-**Integer**, אבל אם נגיד שאחד מהם הוא **Double**, השאר יעקבו אחריו וזה יהפוך לפעולת **Double** וזה ישמור את הערכים העשרוניים שלנו.

הדבר הגדול כאן עם **Double** הוא שהם מחזיקים נקודה עשרונית, ובגלל שהם כן, זה הסוג המספרי הכי בשימוש כשזה מגיע למתמטיקה.

עם **Double** אנחנו יכולים **לחבר**, **לחסר**, **כפל**, **לחלק**, **שורש ריבועי**, כל מה שאנחנו רוצים לעשות בבסיס אנחנו יכולים לעשות עם **Double**, אז מה שלא יגיע לעשות במתמטיקה, נראה **Double**.

Math ב- .Net:

כתבו: **Math**.

וזה יפתח את הספרייה.

כולם פועלים עם **Double**, כי זה הסוג הכי נפוץ בשימוש להחזקת מספרים עם נקודה עשרונית.

זה גם הכי זול, הסיבה היא שזה מעגל את הערך, הם לא אולטרה מדויקים, יש קצת עיגול ב-**Double** שלנו, יש קצת עיגול בכל סוג, זה פשוט איך זה עובד, אבל זה קצת יותר יעיל מסוגים אחרים כי יש לו קצת פחות ספרות משמעותיות, אנחנו יכולים להשתמש בזה כמעט לכל פעולה כשזה מגיע למספרים עם נקודה עשרונית, החרג יהיה **כסף**, אנחנו לא משתמשים ב-**Double** לאחסן כסף כי אנחנו רוצים דיוק יותר גבוה כשזה מגיע לכסף.

```
averageAge = 2.3 / 3;
```

בקטע הזה

```

Console.WriteLine("Hello User! Please write 2 numbers.");
string? numberOneText = Console.ReadLine();
string? numberTwoText = Console.ReadLine();

bool isNumberOneValid = int.TryParse(numberOneText, out int
numberOne);
bool isNumberTwoValid = int.TryParse(numberTwoText, out int
numberTwo);

Console.WriteLine($"This is valid: {isNumberOneValid}, the
number was {numberOne} and this is valid: {isNumberTwoVali
d}, the number was {numberTwo}");

if (isNumberOneValid && isNumberTwoValid)
{
    int sum = numberOne + numberTwo;
    int difference = numberOne - numberTwo;
    double average = (double)numberOne / numberTwo;

    Console.WriteLine($"Sum: {sum}, Difference: {differenc
e}, Average: {average}");
}

```

ה-**Double** נכנס רק בשורה אחת:

```
double average = (double)numberOne / numberTwo;
```

עשינו כאן **Cast (המרה)** מפורשת - לקחנו את **numberOne** שהוא **int**, והמרנו אותו ידנית ל-**Double** באמצעות **(double)** לפני החלוקה.

למה זה נדרש? כי **numberOne** ו-**numberTwo** הם שניהם **int**, וכשמחלקים **int**

ב-C#, ה-**int** עושה **חלוקת שלמים** (Integer Division) - כלומר התוצאה הייתה יוצאת מספר שלם בלבד, בלי נקודה עשרונית, וזה היה גורם לאובדן מידע (**למשל 7/2 היה נותן 3 במקום 3.5**).

ברגע שהמרנו רק את **numberOne** ל-**Double**, כל הפעולה נגררת להיות פעולת **Double** (בדיוק כמו שראינו קודם עם ה-43.0), אז התוצאה נשמרת עם נקודה עשרונית ומאוחסנת במשתנה **average** שהוגדר כ-**Double**.